

useCallback Hook

- The useCallback hook is a performance optimization mechanism in React that helps prevent unnecessary re-renders of child components when their parent component re-renders.
- It achieves this by memoizing (caching) callback functions based on their dependencies.
- If the dependencies haven't changed, the same function reference is returned, avoiding the creation of a new function object on every render. In simpler terms, it helps you avoid re-creating functions on every render, which is especially useful when passing functions as props to child components that rely on the same function reference to avoid unnecessary re-renders.
- In JavaScript, a callback function is a function that is passed as an argument to another function and is executed after some event or action has occurred.
- Callback functions work the same way in the React context as in JavaScript, with the addition of being passed as props to child components.
- This allows children components to communicate with the parent component, enabling a flow of data and actions up the component tree.
- Callback functions are an integral part of React apps as they perform activities like asynchronous operations (e.g., network requests) or handle events (e.g., clicks, form submissions, or other user interactions).
- For example, a button's onClick event handler may be defined as a callback function that updates the state of a React component or performs some action when the button is clicked.

Example:

```
import React from 'react'
import { useState } from 'react'

function Button() {
  const [count, setCount] = useState(0)

  const handleClick = ()=>{
    console.log("Button clicked ...")
```

```

        setCount(count + 1)
      }
    return (
      <div>
        <h1>Count: {count}</h1>
        <button onClick={handleClick}>Increase Count</button>
      </div>
    )
  }

```

export default Button

- In this example, handleClick is a callback function that increments the count state whenever the button is clicked.
- Despite the usefulness of callback functions in React development, especially in handling events and interactions, they can lead to performance issues if not handled properly.
- Every time count changes and Button Component re-renders, a new handleClick function is created.
- From React's perspective, this new function is different from the current render of the previous one, even though its functionality hasn't changed.

Syntax

```

const memoizedCallback = useCallback(() => {
  // Function logic here
}, [dependencies]);

```

Use Cases of useCallback

1. Prevent Unnecessary Re-creation of Functions
 - In React, functions get re-created on every render. If we pass a function as a prop to a child component, it causes unnecessary re-renders of the child. useCallback prevents this by ensuring the function reference remains the same between

renders.

2. Performance Optimization in Large Components

- Reducing unnecessary re-renders and function re-creation helps optimize performance, especially in large applications.

3. Usage with useEffect Dependencies

- If a function is used inside useEffect, every render without useCallback would cause useEffect to run again. useCallback prevents this behavior.

4. Memoized Callbacks in Event Handlers

- Useful when passing event handlers to deeply nested components to avoid unnecessary updates.

Example 1: Button Click Handler Without useCallback

```
import React from 'react'
import { useState } from 'react'

function Button() {
  const [count, setCount] = useState(0)

  const handleClick = ()=>{
    console.log("Button clicked ...")
    setCount(count + 1)
  }
  console.log("Component recreated...")
  return (
    <div>
      <h1>Count: {count}</h1>
      <button onClick={handleClick}>Increase Count</button>
    </div>
  )
}
```

export default Button

- The increment function gets re-created on every render.
- Not a big issue in small components but can cause performance problems in large ones.

With useCallback

```
import React from 'react'
import { useCallback } from 'react'
import { useState } from 'react'

function Button() {
  const [count, setCount] = useState(0)
  const handleClick = useCallback(()=>{
    setCount(count +1)
  }, [count])

  console.log("Component reference is stable")
  return (
    <div>
      <h1>Count: {count}</h1>
      <button onClick={handleClick}>Increase Count</button>
    </div>
  )
}

export default Button
```

Example 2: Usage of useCallback with useEffect Dependencies

- If a function is used inside useEffect, every render without useCallback would cause useEffect to run again because functions in JavaScript are objects and get re-created on every render.

- By using useCallback, we ensure that the function reference remains stable, preventing unnecessary re-executions of useEffect.

Without useCallback

```
import React from 'react'
import { useEffect } from 'react'
import { useState } from 'react'

function Fetching() {
  const [count, setCount] = useState(0)

  const fetchData = () => {
    console.log("Fetching data ...")
  }
  useEffect(() => {
    fetchData();
  }, [fetchData])

  return (
    <div>
      <p>Count: {count}</p>
      <button onClick={() => setCount(count + 1)}>Increase Count</button>
    </div>
  )
}

export default Fetching
```

- Since fetchData is re-created on every render, useEffect re-runs every time the component renders, even when count changes.

- This is unnecessary and inefficient.

With useCallback

```
import React from 'react'
import { useCallback } from 'react'
import { useEffect } from 'react'
import { useState } from 'react'

function Fetching() {
  const [count, setCount] = useState(0)

  const fetchData = useCallback(() => {
    console.log("Fetching data ...")
  }, [])

  useEffect(() => {
    fetchData();
  }, [fetchData])

  return (
    <div>
      <p>Count: {count}</p>
      <button onClick={() => setCount(count + 1)}>Increase Count</button>
    </div>
  )
}
```

export default Fetching

- fetchData is memoized with useCallback, meaning its reference remains stable.
- useEffect now only runs once on mount and doesn't get triggered by re-renders caused by state updates (like changing count).

Example 3: Preventing Unnecessary Function Recreation while passing props from parent to child components

Without useCallback

Parent.jsx

```
import React from 'react'
import { useState } from 'react'
import Child from './Child'

function Parent() {
  const [count, setCount] = useState(0)
  const handleClick = ()=>{
    console.log("Button Clicked")
  }
  return (
    <div>
      <p>Count: {count}</p>
      <button onClick={()=>setCount(count+1)}>Increase Count</button>
      <Child onClick={handleClick}/>
    </div>
  )
}

export default Parent
```

Child.jsx

```
import React from 'react'

function Child({onClick}) {
  console.log("Function reference changed on the parent component...")
  return (
    <div>
```

```

    <button onClick={onClick}>Click Me</button>
  </div>
)
}

```

export default Child

- Every time Parent component re-renders (e.g., when count updates), the handleClick function is recreated.
- Even though Child component still works fine, React sees a new function reference each time, which could cause unnecessary computations in larger applications.

With useCallback

```

import React from 'react'
import { useCallback } from 'react'
import { useState } from 'react'
import Child from './Child'

function Parent() {
  const [count, setCount] = useState(0)
  const handleClick = useCallback(() => {
    console.log("Button clicked!")
  }, [])
  console.log("Parent rendered...")
  return (
    <div>
      <p>Count: {count}</p>
      <button onClick={() => setCount(count+1)}>Increase Count</button>
      <Child onClick={handleClick}/>
    </div>
  )
}

```


export default Parent

- handleClick is now memoized using useCallback(), meaning it does not get recreated unless its dependencies change.
- When the Parent component re-renders (due to count changes), the function reference stays the same.
- The Child component still re-renders, but the function reference remains stable, reducing unnecessary computations.

To Check If a Function is Being Recreated - Using useRef()

- You can store the function in useRef() and check if the reference changes.

```
import React from 'react'
```

```
import { useCallback } from 'react'
```

```
import { useState, useRef } from 'react'
```

```
import Child from './Child'
```

```
function Parent() {
```

```
  const [count, setCount] = useState(0)
```

```
  // Store previous function reference
```

```
  const prevFunctionRef = useRef(null)
```

```
  // const handleClick = ()=>{
```

```
    // console.log("Button Clicked")
```

```
  // }
```

```
  const handleClick = useCallback(()=>{
```

```
    console.log("Button clicked!")
```

```
  }, [])
```

```
  console.log("Parent rendered...")
```

```
  // Check if function reference changed
```

```
  console.log("Function changed:", prevFunctionRef.current !== handleClick)
```

```

    prevFunctionRef.current = handleClick
  return (
    <div>
      <p>Count: {count}</p>
      <button onClick={()=>setCount(count+1)}>Increase Count</button>
      <Child onClick={handleClick}/>
    </div>
  )
}
export default Parent

```

When NOT to Use useCallback?

- If the function is simple and doesn't cause performance issues.
- If the function is not being passed as a prop to a child component.
- If memoization adds unnecessary complexity without significant performance gain.

Differences Between useCallback and useMemo

1. Purpose:
 - useCallback is used to memoize functions so that they don't get recreated on every render.
 - useMemo is used to memoize computed values to avoid unnecessary recalculations.
2. What It Returns:
 - useCallback returns a function that only changes when its dependencies change.
 - useMemo returns a computed value that only recomputes when its dependencies change.
3. Use Case:

- Use `useCallback` when you want to pass a function as a prop to a child component and prevent unnecessary re-renders.
- Use `useMemo` when you have an expensive calculation (like filtering, sorting, or computing derived values) that you don't want to recompute unnecessarily.

4. When It Runs:

- `useCallback` runs when the component renders but only creates a new function when dependencies change.
- `useMemo` runs when the component renders but only recomputes the value if dependencies change.